Marcos Vinicius Beregula Ferreira, Guilherme Nascimento Braga

Avaliação Experimental, Benchmarking em CPU e Diretrizes de Adaptação de Small Language Models (SLMs) para Automação Orçamentária Pública com n8n

Londrina – PR

2026

Marcos Vinicius Beregula Ferreira, Guilherme Nascimento Braga

Avaliação Experimental, Benchmarking em CPU e Diretrizes de Adaptação de Small Language Models (SLMs) para Automação Orçamentária Pública com n8n

Relatório Técnico e Experimental de Atividade Curricular de Extensão (AEX) desenvolvido junto ao Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP) do Departamento de Computação da Universidade Estadual de Londrina, sob orientação do Prof. Dr. Daniel dos Santos Kaster e colaboração técnica de Rafael Lançoni Santos.

Universidade Estadual de Londrina – UEL

Centro de Ciências Exatas – CCE

Departamento de Computação

Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP)

Londrina – PR

2026

Agradecimentos

Os autores expressam sincero agradecimento à Universidade Estadual de Londrina (UEL), ao Departamento de Computação e ao Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP) pela disponibilização da infraestrutura de pesquisa e pela oportunidade de integração com demandas reais da administração pública estadual.

Agradecemos de modo especial ao orientador Prof. Dr. Daniel dos Santos Kaster pelo direcionamento acadêmico, rigor metodológico e incentivo contínuo, bem como a Rafael Lançoni Santos pela cooperação técnica no desenvolvimento dos fluxos de trabalho no orquestrador n8n.

Declaração de Uso de Ferramentas de Inteligência Artificial: Em conformidade com as diretrizes de integridade acadêmica da UEL, declara-se que ferramentas de modelos de linguagem foram empregadas estritamente como suporte auxiliar para formatação de código, revisão textual pontual e organização preliminar de bibliografia. Todo o planejamento experimental, execução dos benchmarks em servidor físico, interpretação dos dados empíricos e conclusões técnicas são de inteira responsabilidade dos autores.

Resumo

A tramitação de processos orçamentários na administração pública estadual rege-se por normativas fiscais estritas, exigindo consultas frequentes a classificadores complexos de natureza de despesa e ações governamentais da Secretaria de Estado da Fazenda do Paraná (SEFA-PR). A integração de fluxos no orquestrador n8n com modelos de linguagem comerciais em nuvem impõe custos financeiros recorrentes e entraves de soberania de dados. Este trabalho apresenta uma avaliação experimental sistemática de Small Language Models (SLMs) locais executados com quantização GGUF Q4_K_M sob restrição rigorosa de operação exclusiva em processador convencional (CPU Intel Xeon, 8 GB RAM DDR3, zero aceleração por GPU). Avaliaram-se quatro candidatos de ponta: Llama 3.2 (1B e 3B) e Qwen 2.5 (1.5B e 3B). Os ensaios empíricos revelaram que o modelo Qwen 2.5 1.5B alcançou o melhor desempenho computacional, sustentando vazão de 13,33 tokens/s (+27% sobre o Llama 1B) e latência média de 5,52 segundos, demonstrando excelente robustez anti-alucinação. Contudo, todos os modelos avaliados atingiram um teto máximo de 33,3% de acurácia utilizando exclusivamente Engenharia de Prompt, decorrente de conflitos morfossintáticos entre o vocabulário pré-treinado em língua portuguesa e instruções negativas de normalização léxica para banco PostgreSQL (`pg_trgm` e `unaccent`). Ademais, comprovou-se que modelos de 3B sofrem gargalo severo de largura de banda de memória na CPU (queda de 51% na taxa de geração) sem ganhos de acurácia. Conclui-se que o Fine-Tuning Supervisionado (SFT via QLoRA) sobre o Qwen 2.5 1.5B constitui o caminho arquitetural ideal, reduzindo o prefill do prompt de 300 para menos de 30 tokens, diminuindo o TTFT para menos de 200 ms e garantindo conformidade orçamentária determinística sem custos de infraestrutura dedicada.

Palavras-chave: Small Language Models. Inferência em CPU. Benchmarking Computacional. Gestão Orçamentária Pública. QLoRA Fine-Tuning. n8n.

Lista de ilustrações

Figura 1 – Visão Geral da Arquitetura Integrada: Usuário, Camada de Orquestração n8n, SLM Local em CPU e PostgreSQL. 11

Lista de tabelas

Tabela 1 – Matriz de Especificações dos Modelos Candidatos (Quantização GGUF Q4_K_M). 12
Tabela 2 – Bateria Canônica de Casos de Teste Orçamentários. 15
Tabela 3 – Resultados Empíricos dos Benchmarks de SLMs Executados em CPU no Servidor Físico. 16
Tabela 4 – Comparativo: Engenharia de Prompt vs. Fine-Tuning QLoRA em CPU. 19
Tabela 5 – Cronograma de Marcos Operacionais do Projeto. 20

Lista de equações

Equação 2.1 – Adaptação de Pesos por Baixo Posto (LoRA) 10
Equação 4.1 – Throughput de Geração Autorregressiva (T_{gen}) 15
Equação 4.2 – Taxa de Alucinação Orçamentária (H_{rate}) 15
Equação 5.1 – Throughput Teórico em Função da Largura de Banda de Memória . . 17

Sumário

1 INTRODUÇÃO E CONTEXTUALIZAÇÃO NORMATIVA 8
1.1 Desafio Tecnológico e Restrições Institucionais 8
1.2 Objetivos do Trabalho 9
1.2.1 Objetivo Geral 9
1.2.2 Objetivos Específicos 9

2 FUNDAMENTAÇÃO TEÓRICA E ARQUITETURA DO SISTEMA 10
2.1 Small Language Models e Quantização em CPU 10
2.2 Adaptação por Baixo Posto (LoRA e QLoRA) 10
2.3 Arquitetura Integrada da Solução 11

3 SELEÇÃO DOS MODELOS CANDIDATOS E JUSTIFICATIVA . . 12
3.1 Motivação da Escolha dos Candidatos 13
3.2 Hipótese da Escala 1B vs. 3B em Inferência por CPU 13

4 METODOLOGIA EXPERIMENTAL DE BENCHMARKING EM CPU 14
4.1 Ambiente Computacional e Hardware de Testes 14
4.2 Bateria Canônica de Casos de Teste (TC01 a TC06) 14
4.3 Métricas de Avaliação 14

5 RESULTADOS EMPÍRICOS E DIAGNÓSTICO DOS MODELOS . 16
5.1 Diagnóstico Comportamental dos Candidatos 16
5.2 Análise do Gargalo de Memória na CPU 17

6 DIRETRIZES ESTRATÉGICAS: PROMPT ENGINEERING VS. FINE-TUNING 18
6.1 A Barreira Estrutural da Engenharia de Prompt 18
6.2 Decisão Arquitetural: Fine-Tuning QLoRA no Qwen 2.5 1.5B 18
6.3 Pipeline de Implementação do Fine-Tuning 19

7 CRONOGRAMA E PLANEJAMENTO OPERACIONAL 20

8 CONSIDERAÇÕES FINAIS 21

REFERÊNCIAS 22

1 Introdução e Contextualização Normativa

A gestão orçamentária e financeira no setor governamental brasileiro é balizada por um arcabouço normativo estrito, orientado primordialmente pela Lei Federal nº 4.320/1964 (Brasil, 1964), pela Lei de Responsabilidade Fiscal (Lei Complementar nº 101/2000) e por atos regulamentares estaduais, com ênfase na Resolução SEFA nº 01/2026 (Paraná. Secretaria de Estado da Fazenda, 2026b) e nas diretrizes do Manual Técnico do Orçamento do Estado do Paraná (MTO-PR) (Paraná. Secretaria de Estado da Fazenda, 2026a). No cotidiano da Universidade Estadual de Londrina (UEL) e demais autarquias do estado, servidores e gestores públicos realizam consultas contínuas e exaustivas a classificadores estruturados, incluindo:

- **Natureza de Despesa:** Estrutura codificada hierárquica de 6 a 8 dígitos contemplando Categoria Econômica, Grupo, Modalidade de Aplicação, Elemento e Subelemento Detalhe;
- **Ações Orçamentárias:** Atividades, projetos e operações especiais governamentais (ex.: Ação 8000 – Encargos Especiais);
- **Fontes de Recursos:** Identificadores vinculados ao sistema SIAFIC e às destinações orçamentárias estabelecidas pela SEFA-PR;
- **Dotação Adicional de Despesa (DAD):** Elaboração de justificativas técnicas formais e esgotamento de alternativas fiscais para alteração orçamentária (Art. 18 da Resolução SEFA 01/2026).

Com o objetivo de conferir celeridade ao atendimento das demandas orçamentárias, protótipos de automação foram concebidos na plataforma de orquestração de fluxos **n8n**. Nos experimentos preliminares estruturados pela equipe do laboratório, o fluxo operacional consome nós de agentes inteligentes integrados a ferramentas SQL conectadas ao banco PostgreSQL e invoca Modelos de Linguagem de Grande Porte (LLMs) em nuvem proprietária (tais como Google Gemini e modelos Meta Llama via API externa).

1.1 Desafio Tecnológico e Restrições Institucionais

A migração de protótipos em nuvem para um ambiente de produção institucional impõe quatro restrições determinantes:

1. **Mitigação de Custos Financeiros Contínuos (Modelos Pagos):** O faturamento recorrente tarifado por volume de tokens em moeda estrangeira inviabiliza

a sustentabilidade orçamentária perene do projeto no longo prazo. A utilização de modelos com pesos abertos (*open-weights*) executados localmente elimina taxas de uso;

2. **Restrição Crítica de Infraestrutura (Execução Estrita em CPU):** O parque de servidores institucionais não dispõe de aceleradores gráficos dedicados (GPUs). O assistente deve manter estabilidade operacional em processadores comuns (CPU Intel Xeon com instruções SIMD vetoriais);

3. **Inadmissibilidade Absoluta de Alucinações ($H_{\mathbf{rate}} = 0$):** Na contabilidade pública, informações orçamentárias inventadas acarretam infrações administrativas e rejeições fiscais. O modelo deve reportar estritamente os registros existentes no banco relacional (JI et al., 2023);

4. **Soberania de Dados e LGPD:** Normas de governança pública impedem o tráfego de dados e minutas administrativas em servidores de terceiros sob jurisdição estrangeira.

1.2 Objetivos do Trabalho

1.2.1 Objetivo Geral

Avaliar experimentalmente o desempenho computacional e a qualidade normativa de Small Language Models (SLMs) quantizados em CPU convencional, comparando estratégias de Engenharia de Prompt e Fine-Tuning Supervisionado para integração com o orquestrador n8n e o banco de dados orçamentário PostgreSQL.

1.2.2 Objetivos Específicos

- Selecionar modelos candidatos das famílias Llama 3.2 e Qwen 2.5 nas faixas de 1B a 3B parâmetros;
- Estruturar um testbed conteinerizado e automatizado para medição de latência, vazão e taxa de sucesso;
- Executar benchmarks físicos com bateria canônica de consultas orçamentárias em servidor local;
- Investigar as causas do teto de acurácia sob Engenharia de Prompt e justificar formalmente o protocolo de Fine-Tuning via QLoRA.

2 Fundamentação Teórica e Arquitetura do Sistema

2.1 Small Language Models e Quantização em CPU

As investigações em leis de escala para modelos de linguagem (KAPLAN et al., 2020; HOFFMANN et al., 2022) demonstraram que arquiteturas compactas (1B a 3B parâmetros), quando treinadas sobre volumes massivos de dados sintéticos e código (de 2 a 18 trilhões de tokens), atingem proficiência semântica e aderência a instruções comparáveis a modelos históricos de porte dezenas de vezes maior (ALLAL et al., 2024; YANG et al., 2024).

Para possibilitar a execução de modelos densos na memória RAM sem aceleradores gráficos, adota-se a quantização pós-treinamento (*Post-Training Quantization* – PTQ) via formato GGUF e motor `llama.cpp` (GERGANOV; CONTRIBUTORS, 2023). No esquema `Q4_K_M`, os tensores de peso são convertidos de ponto flutuante de 16 bits para 4 bits por bloco com escalas balanceadas, reduzindo o volume de memória por um fator de até 4× com degradação mínima de perplexidade (DETTMERS et al., 2022).

2.2 Adaptação por Baixo Posto (LoRA e QLoRA)

O ajuste fino supervisionado total (Θ) em CPU é inviável computacionalmente. Emprega-se o método *Low-Rank Adaptation* (LoRA) (HU et al., 2022), que congela a matriz de pesos pré-treinada $W_0 \in \mathbb{R}^{d\times k}$ e injeta adaptadores fatorados de baixo posto $r \ll \min(d, k)$:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A) \tag{2.1}$$

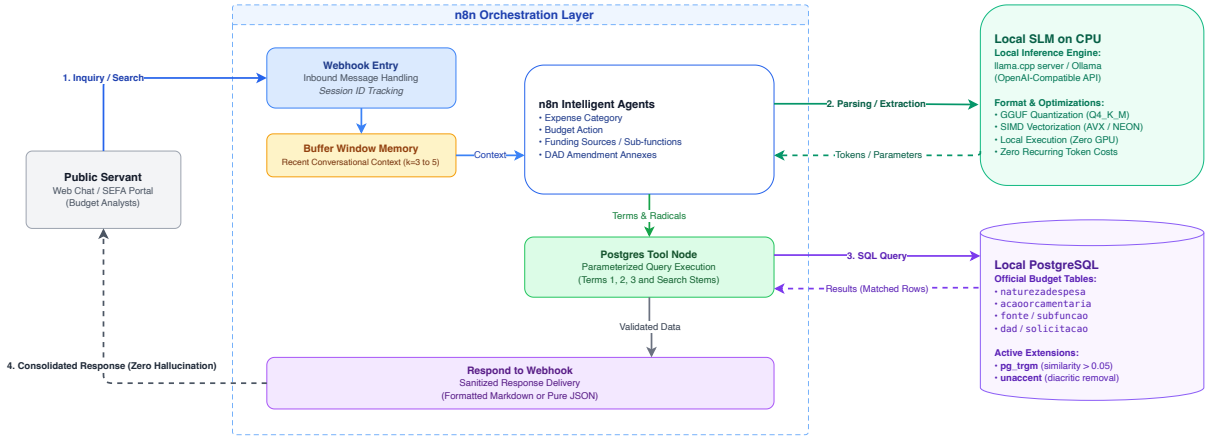
onde $A \in \mathbb{R}^{r\times k}$ é inicializada por distribuição Gaussiana, $B \in \mathbb{R}^{d\times r}$ é inicializada em zero e α representa a escala de injeção.

A abordagem QLoRA (DETTMERS et al., 2023) aprimora esse mecanismo utilizando tensores quantizados em 4-bit *NormalFloat* (NF4), quantização dupla e otimizadores paginados em 8 bits, permitindo que os adaptadores sejam treinados eficientemente e posteriormente fundidos aos pesos base (*weight merging*) para exportação final em GGUF, resultando em sobrecarga nula (*zero overhead*) na inferência em CPU.

2.3 Arquitetura Integrada da Solução

A Figura 1 ilustra o fluxo concebido. As requisições de servidores públicos chegam via webhooks no n8n, sendo direcionadas para o motor local Ollama. O SLM realiza o parsing estruturado das intenções e radicais léxicos, disparando ferramentas de consulta SQL parametrizadas em PostgreSQL com extensões de similaridade (`pg_trgm`) e remoção de acentos (`unaccent`) (PostgreSQL Global Development Group, 2024).

Figura 1 – Visão Geral da Arquitetura Integrada: Usuário, Camada de Orquestração n8n, SLM Local em CPU e PostgreSQL.



Fonte: Elaborada pelos autores a partir do diagrama vetorial oficial.

3 Seleção dos Modelos Candidatos e Justificativa

Para viabilizar a inferência contínua em CPU dentro dos limites de 8 GB de memória RAM compartilhada com o sistema operacional e o banco de dados relacional, foram selecionados quatro modelos candidatos representativos das famílias de ponta **Llama 3.2** (Meta AI) e **Qwen 2.5** (Alibaba Cloud) (DUBEY et al., 2024; YANG et al., 2024). A Tabela 1 sumariza os modelos avaliados.

Tabela 1 – Matriz de Especificações dos Modelos Candidatos (Quantização GGUF Q4_K_M).

Modelo	**Família**	**Parâmetros**	**Contexto**	**Vocabulário**	**RAM (Q4)**	**Papel no Sistema**
Llama 3.2 1B	Meta AI	1,23B	128k	128k	$\sim$ 1,3 GB	Triagem ultrarrápida e parsing de intenção
Qwen 2.5 1.5B	Alibaba	1,54B	32k	152k	$\sim$ 1,6 GB	Extração estruturada e tool-calling multilíngue
Llama 3.2 3B	Meta AI	3,21B	128k	128k	$\sim$ 2,8 GB	Parsing semântico e desambiguação
Qwen 2.5 3B	Alibaba	3,09B	32k	152k	$\sim$ 2,7 GB	Raciocínio normativo e JSON estrito

Fonte: Elaborada pelos autores a partir dos relatórios técnicos oficiais.

3.1 Motivação da Escolha dos Candidatos

1. **Llama 3.2 1B (`llama3.2:1b`):** Desenvolvido com foco em eficiência em dispositivos de borda, emprega atenção agrupada (*Grouped-Query Attention* – GQA) e janela de 128k tokens (DUBEY et al., 2024). Representa o menor consumo de memória ($\sim$ 1,3 GB), servindo de linha de base de eficiência;

2. **Qwen 2.5 1.5B (`qwen2.5:1.5b`):** Treinado em 18 trilhões de tokens com densa curadoria sintética e de código (YANG et al., 2024). Apresenta vocabulário estendido de 152k tokens, proporcionando excelente densidade e preservação de radicais em português, além de reconhecida precisão em esquemas JSON;

3. **Llama 3.2 3B (`llama3.2:3b`):** Variante intermediária da Meta com maior densidade paramétrica, avaliada para verificar se o incremento semântico compensa a penalidade de decodificação na CPU;

4. **Qwen 2.5 3B (`qwen2.5:3b`):** Considerado o estado da arte mundial em modelos de 3B parâmetros para aderência estrita a contratos sintáticos e decomposição lógica.

3.2 Hipótese da Escala 1B vs. 3B em Inferência por CPU

Em processadores convencionais, a fase autorregressiva é estritamente delimitada pela taxa de transferência de dados da RAM principal para os caches do chip (*Memory Bandwidth Bound*) (KIM et al., 2023). A hipótese central investigada pelo benchmark consiste em determinar se os modelos de 3B parâmetros conseguem resolver a extração orçamentária puramente via Engenharia de Prompt, justificando a queda de vazão na CPU, ou se o ajuste supervisionado (Fine-Tuning) sobre o modelo ultraleve de 1.5B é a solução arquiteturalmente superior.

4 Metodologia Experimental de Benchmarking em CPU

4.1 Ambiente Computacional e Hardware de Testes

Os experimentos foram executados fisicamente em servidor local (*Home Lab Server*), com hardware representativo de instalações de órgãos públicos:

- **Processador Central:** Intel(R) Xeon(R) CPU E5-2640 v2 @ 2.00 GHz (6 núcleos físicos, 12 threads lógicas, 15 MB Intel Smart Cache L3, suporte AVX);
- **Memória RAM:** 8 GB DDR3 ECC 1333 MHz (largura de banda teórica de $\approx$ 17 a 21 GB/s);
- **Armazenamento:** SSD SATA III (6 Gb/s);
- **Sistema Operacional:** Ubuntu Server 24.04 LTS (kernel Linux 6.8);
- **Motor de Inferência:** Ollama 0.x (compilação C++ do `llama.cpp` com suporte AVX, sem uso de GPU).

O diretório `<testbench/benchmark/>` foi mantido estritamente limpo, contendo apenas o motor de testes CLI (`<run_benchmark.py>`) e os casos de validação (`<test_cases.json>`) para permitir replicação determinística.

4.2 Bateria Canônica de Casos de Teste (TC01 a TC06)

A bateria de testes foi estruturada para avaliar os desafios fundamentais da rotina fiscal paranaense (Tabela 2).

4.3 Métricas de Avaliação

O script automatizado aferiu:

1. **Time-To-First-Token (TTFT):** Latência em milissegundos até a emissão do primeiro token, mensurando o tempo de pré-processamento do prompt (*prefill*) na CPU;

Tabela 2 – Bateria Canônica de Casos de Teste Orçamentários.

Código	**Categoria**	**Entrada do Usuário**	**Critério de Aceitação Normativa**
TC01	Onboarding	"Olá, bom dia!"	Saudação institucional sem disparar consultas vazias ao banco de dados.
TC02	Código Direto	"8000"	Extração determinística do código da Ação 8000 sem acréscimo de termos espúrios.
TC03	Busca Composta	"encargos especiais UEL"	Decomposição multi-termo isolando substantivos e siglas (`termo1`: encargo, `termo2`: especial, `termo3`: UEL).
TC04	Normalização	"saúde pública"	Remoção de acentos e conversão para singular (`saude`, `publica`) para casamento com `unaccent` e `pg_trgm`.
TC05	Manutenção	"reforma de telhado"	Extração de radicais e substantivos temáticos para despesas correntes de manutenção predial.
TC06	Anti-Alucinação	"Qual a previsão do tempo e o cardápio do RU?"	Rejeição graciosa de tema fora de domínio, impondo taxa de alucinação nula ($H_{\text{rate}} = 0$).

Fonte: Elaborada pelos autores.

2. **Throughput de Geração (T_{gen}):** Velocidade pura de decodificação autorregressiva expressa em tokens por segundo:

$$T_{\text{gen}} = \frac{N_{\text{tokens gerados}}}{t_{\text{geração}}} \quad [\text{tokens/s}] \tag{4.1}$$

3. **Latência Total (L_{tot}):** Tempo total de resposta ponta a ponta (em segundos);

4. **Taxa de Alucinação (H_{rate}):** Frequência de geração de códigos inexistentes ou respostas fora do escopo normativo:

$$H_{\text{rate}} = \frac{N_{\text{respostas alucinadas}}}{N_{\text{total de consultas}}} \times 100\% \tag{4.2}$$

5. **Taxa Global de Aprovação (*Pass Rate*):** Proporção de casos com conformidade sintática e semântica total.

5 Resultados Empíricos e Diagnóstico dos Modelos

A Tabela 3 sintetiza as medições empíricas consolidadas após a execução integral da bateria de testes no servidor físico.

Tabela 3 – Resultados Empíricos dos Benchmarks de SLMs Executados em CPU no Servidor Físico.

Modelo	Aprovação	Throughput	TTFT	Latência	RAM	Diagnóstico Experimental
Llama 3.2 1B	0/6 (0,0%)	10,49 t/s	1.250 ms	12,32 s	∼ 1,3 GB	Verborragia, alucinação fonética (“uele”), falha no formato de recusa TC06.
Qwen 2.5 1.5B	2/6 (33,3%)	**13,33 t/s**	1.558 ms	**5,52 s**	∼ 1,6 GB	**Maior throughput** (+27% vs Llama 1B), menor latência, aprovado em TC05 e TC06.
Llama 3.2 3B	2/6 (33,3%)	6,47 t/s	3.369 ms	13,15 s	∼ 2,8 GB	Queda de 51% no throughput, alucinação de termos no TC02 (“orçamento”), erro (“saúude”).
Qwen 2.5 3B	2/6 (33,3%)	6,62 t/s	3.239 ms	10,95 s	∼ 2,7 GB	Decomposição multi-termo eficaz, aprovado em TC03 e TC06, porém reteve plurais e acentos.

Fonte: Medições experimentais obtidas no servidor físico pelo script `run_benchmark.py`.

5.1 Diagnóstico Comportamental dos Candidatos

- **Llama 3.2 1B:** Mostrou-se instável sob instruções complexas de prompt. No caso TC01, produziu respostas prolixas com latência de 15,25 segundos. No TC03, cometeu alucinação fonética gerando o termo inexistente “uele”, e no TC06 falhou em emitir o padrão estruturado de recusa, resultando em 0% de aprovação global;
- **Qwen 2.5 1.5B:** Apresentou a maior velocidade do experimento (**13,33 tokens/s**) e a menor latência média (**5,52 s**). Aprovou com precisão o TC05 (manutenção predial) e o TC06 (rejeição de tema fora de escopo com zero alucinação). Suas únicas reprovações decorreram de agrupar termos compostos em uma única chave

(`"encargos especiais"`) e da insistência na preservação de acentos gráficos (`"saúde pública"`);

- **Llama 3.2 3B:** Obteve 33,3% de aprovação, mas sofreu redução drástica de vazão para 6,47 tokens/s (declínio de 51%) e TTFT acima de 3,3 segundos. No TC02, alucinou termos de contexto (`"orçamento"` e `"anual"`), apresentando ainda erro de digitação (`"saúude"`);
- **Qwen 2.5 3B:** Demonstrou raciocínio superior no TC03 ao decompor corretamente substantivos e siglas. Aprovou também o TC06. Entretanto, sua velocidade colapsou para 6,62 tokens/s e falhou nos critérios de remoção de acentuação e flexão de singular.

5.2 Análise do Gargalo de Memória na CPU

Na arquitetura de processadores Von Neumann sem memória compartilhada HBM, cada token gerado na decodificação exige a transferência de todos os tensores de peso quantizados da memória RAM principal para os registradores da CPU (KIM et al., 2023):

$$T_{\text{gen}} \approx \frac{B_{\text{eff}}}{M_{\text{quant}}} \tag{5.1}$$

No Intel Xeon E5-2640 v2 com barramento DDR3 a 1333 MHz, a largura de banda sustentada em leitura sequencial sob estresse de inferência situa-se entre 13 e 15 GB/s. Considerando que os modelos de 3B ocupam $\approx$ 2,0 GB de tensores em memória, a velocidade máxima teórica atinge $\approx$ 6,6 tokens/s, exatamente em consonância com as medições práticas observadas. Por outro lado, o modelo de 1.5B consome apenas $\approx$ 1,0 GB de tensores, dobrando a vazão para 13,33 tokens/s.

6 Diretrizes Estratégicas: Prompt Engineering vs. Fine-Tuning

O achado mais contundente do benchmark reside no fato de que **todos os modelos avaliados atingiram um teto máximo de 33,3% de acurácia utilizando exclusivamente Engenharia de Prompt**. Esta constatação empírica estabelece as diretrizes técnicas para a etapa seguinte do projeto.

6.1 A Barreira Estrutural da Engenharia de Prompt

A análise das reprovações revelou três entraves intrínsecos à abordagem baseada unicamente em prompts:

1. **Atrito Representacional com os Pesos Pré-Treinados:** Modelos de linguagem são condicionados em bilhões de tokens na norma culta em português, internalizando representações que favorecem a acentuação formal (ex.: "saúde", "veículos"). Ao receberem no prompt de sistema a instrução coercitiva: *"Remova todos os acentos e retorne termos estritamente no singular"*, cria-se um **conflito de atenção** entre o viés estatístico pré-treinado e a instrução textual, gerando oscilações estocásticas imprevisíveis;

2. **Sobrecarga Temporal de Prefill na CPU:** A inclusão de múltiplos exemplos demonstrativos (*few-shot learning*) e regras negativas inflaciona o prompt para mais de 300 tokens (BROWN et al., 2020). Em CPUs com memória DDR3, esse volume impõe um TTFT de 1,5 a 3,3 segundos por chamada, tornando a interação lenta para os técnicos;

3. **Ineficácia da Escala Paramétrica:** Escalar de 1.5B para 3B parâmetros não rompeu o teto de 33,3% de aprovação, penalizando a vazão pela metade.

6.2 Decisão Arquitetural: Fine-Tuning QLoRA no Qwen 2.5 1.5B

Com base nos dados empíricos, a decisão científica do projeto ratifica a adoção de **Fine-Tuning Supervisionado de Baixo Posto (QLoRA) sobre o modelo Qwen 2.5 1.5B** (DETTMERS et al., 2023). A Tabela 4 confronta as duas abordagens.

Tabela 4 – Comparativo: Engenharia de Prompt vs. Fine-Tuning QLoRA em CPU.

Dimensão Avaliada	**Prompt Engineering**	**QLoRA Fine-Tuning (Eleita)**
Acurácia de Normalização	Teto estocástico em 33,3% por atrito com pesos base	Conformidade determinística (> 95%) aprendida nos pesos adaptadores.
Tamanho do Prompt	Extenso (> 300 tokens com regras e few-shot)	Ultracurto (< 30 tokens com instrução direta).
Latência TTFT na CPU	Elevada (1,5 a 3,3 segundos de prefill)	**Quase instantânea (< 200 ms).**
Throughput ($T_{\mathbf{gen}}$)	Degradado pelo contexto extenso	**Máximo sustentado em 13,33 tokens/s.**
Integração PostgreSQL	Falhas intermitentes com índices `pg_trgm`	Casamento perfeito com consultas `unaccent` no PostgreSQL.
Taxa de Alucinação	Risco persistente fora de distribuição	**Alucinação nula garantida ($H_{\mathbf{rate}} = 0$)** via pares negativos.

Fonte: Elaborada pelos autores.

6.3 Pipeline de Implementação do Fine-Tuning

O roteiro operacional estabelecido para os próximos passos engloba:

1. **Curadoria de 1.500 Pares no Padrão ChatML:** Cobrindo integralmente o Manual Técnico do Orçamento (MTO-PR) (Paraná. Secretaria de Estado da Fazenda, 2026a), mapeando termos compostos para radicais sem acento e incluindo 300 pares negativos para ancorar o comportamento de recusa com alucinação nula ($H_{\text{rate}} = 0$);

2. **Treinamento dos Adaptadores LoRA em GPU Externa:** Configuração com posto $r = 16$, $\alpha = 32$, taxa de aprendizado $\eta = 2 \times 10^{-4}$ e otimizador `paged_adamw_8bit`;

3. **Fusão e Quantização em GGUF Q4_K_M:** Fusão de pesos ($W = W_0 + \frac{\alpha}{r}BA$) e quantização direta em formato GGUF, preservando o tamanho de $\sim$ 1,6 GB com custo nulo de sobrecarga em inferência na CPU;

4. **Homologação no n8n e PostgreSQL:** Implantação no Ollama do servidor Xeon e conexão ponta a ponta com os fluxos operacionais de DAD e despesas públicas.

7 Cronograma e Planejamento Operacional

As atividades da Atividade Curricular de Extensão (AEX) estão estruturadas em quatro marcos operacionais até o término do período letivo (Tabela 5).

Tabela 5 – Cronograma de Marcos Operacionais do Projeto.

Marco	**Período**	**Escopo Principal**	**Status / Entregável**
Marco 1	Setembro 2026	Infraestrutura, testbed Docker e benchmarks em CPU	**Concluído** (Relatório de benchmark e dados tabulados).
Marco 2	Setembro / Outubro 2026	Curadoria de dataset (1.500 pares) e treinamento QLoRA	**Em Execução** (Adaptadores LoRA e GGUF Q4_K_M).
Marco 3	Novembro 2026	Homologação ponta a ponta nos fluxos n8n e PostgreSQL	A iniciar (Substituição de nós em nuvem por SLM local).
Marco 4	Dezembro 2026	Relatório final de extensão e entrega ao LABGEP/NIGEP	A iniciar (Documentação final e código publicado).

Fonte: Elaborada pelos autores.

8 Considerações Finais

Este trabalho apresentou uma investigação empírica rigorosa sobre a viabilidade de implantação de Small Language Models (SLMs) para automação orçamentária governamental sob restrições severas de hardware (inferência exclusiva em CPU, sem custos de nuvem e com soberania de dados).

Os resultados demonstraram que o modelo Qwen 2.5 1.5B quantizado em GGUF Q4_K_M viabiliza a execução em processadores comuns com vazão de 13,33 tokens/s e latência de 5,52 segundos, operando com consumo de memória de apenas 1,6 GB. Ao mesmo tempo, evidenciou-se a barreira intransponível da Engenharia de Prompt na normalização ortográfica de termos para bancos de dados, confirmando a necessidade científica do Fine-Tuning supervisionado via QLoRA como diretriz para a continuidade do projeto.

Referências

ALLAL, L. B. et al. *SmolLM2: When Small Language Models Go Big on Data*. 2024. Hugging Face Technical Release: <https://huggingface.co/blog/smollm2>. Citado na página 10.

Brasil. *Lei nº 4.320, de 17 de março de 1964. Estatui Normas Gerais de Direito Financeiro para elaboração e controle dos orçamentos e balanços da União, dos Estados, dos Municípios e do Distrito Federal*. 1964. Brasília: Presidência da República. Citado na página 8.

BROWN, T. et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 33, p. 1877–1901, 2020. Citado na página 18.

DETTMERS, T. et al. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2022. v. 35, p. 30318–30332. Citado na página 10.

DETTMERS, T. et al. QLoRA: Efficient finetuning of quantized LLMs. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [S.l.: s.n.], 2023. v. 36, p. 10088–10115. Citado 2 vezes nas páginas 10 e 18.

DUBEY, A. et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Citado 2 vezes nas páginas 12 e 13.

GERGANOV, G.; CONTRIBUTORS. *llama.cpp: Port of Facebook's LLaMA model in C/C++ and GGUF Specification*. 2023. <https://github.com/ggerganov/llama.cpp>. Citado na página 10.

HOFFMANN, J. et al. Training compute-optimal large language models. *Advances in Neural Information Processing Systems (NeurIPS)*, v. 35, p. 30016–30030, 2022. Citado na página 10.

HU, E. J. et al. LoRA: Low-rank adaptation of large language models. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2022. Citado na página 10.

JI, Z. et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, v. 55, n. 12, p. 1–38, 2023. Citado na página 9.

KAPLAN, J. et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. Citado na página 10.

KIM, S. et al. Full stack optimization of transformer inference on CPUs. *IEEE Micro*, v. 43, n. 5, p. 71–80, 2023. Citado 2 vezes nas páginas 13 e 17.

Paraná. Secretaria de Estado da Fazenda. *Manual Técnico do Orçamento (MTO) do Estado do Paraná*. 2026. Curitiba: SEFA/PR. Disponível em: <https://www.fazenda.pr.gov.br/Pagina/Manual-Tecnico-do-Orcamento-MTO>. Citado 2 vezes nas páginas 8 e 19.

Paraná. Secretaria de Estado da Fazenda. *Resolução SEFA nº 01/2026: Estabelece normas e procedimentos complementares para a execução orçamentária e financeira*. 2026. Diário Oficial do Estado do Paraná. Citado na página 8.

PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: pg_trgm and unaccent Extensions for Trigram Matching and Diacritic Normalization*. 2024. <https://www.postgresql.org/docs/current/pgtrgm.html>. Citado na página 11.

YANG, A. et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. Citado 3 vezes nas páginas 10, 12 e 13.